

Super-Precise Latency: How CPU Threading Affects High-Precision Latency, and an Industry-Wide Audit

Gustavo Pedro Ricou and Romaric Duvignau

Abstract—Engineers choose message brokers, which carry data between programs, partly by latency benchmarks that now report latencies below a millisecond. At that scale, when a program reads the clock matters. After a broker writes a message, it acknowledges it to the producer and delivers it to the consumer. On one machine and one clock, the two readings should coincide. Measured, their difference spreads over milliseconds, and part of it falls below zero. The cause is scheduling. A thread must wait for a processor core before it can read the clock, and the thread that reads the acknowledgment can wait longer than the message takes to reach the consumer. How often this happens obeys laws we derive and test by manipulation. It rises with load and with how the load is placed, and falls when the clock-reading threads run at real-time priority and as the timed path outgrows the scheduler’s slice, which the processor count sets. An industry-wide audit finds such values discarded uncounted or never measured, and a filter on millisecond clocks that deletes the fastest messages too. We give checks, and measure what each buys.

Index Terms—Apache Kafka; benchmark auditing; CPU scheduling; latency benchmarking; message brokers; multi-threading; Redis Streams; timestamp resolution.

I. INTRODUCTION

MANY online services move their data through a message broker, a service that takes each message from the program that publishes it, the *producer*, and delivers it to the program that receives it, the *consumer* (Fig. 1a). Engineers choose among brokers partly by latency benchmarks [1], which time each message from producer to consumer and now report latencies below a millisecond [2]. At that scale, a thread’s wait for a processor core can exceed the latency itself. A team that ranks brokers or promises a response time on such a number may be acting on how the number was measured, not on the broker.

We study one difference that, in the ideal case, is zero. Once the broker has written a message, it returns an acknowledgment to the producer and delivers the message to the consumer. Both leave after the same write. When the producer and the consumer run on one machine and read its one clock, the producer’s reading of the acknowledgment, t_{ack} , and the consumer’s reading of the message, t_{recv} , should coincide. We make four contributions.

- 1) **The distribution** (Section III). On one clock, $S = t_{\text{recv}} - t_{\text{ack}}$ is spread over milliseconds, and 8.43% of it

falls below zero. Nothing arrived early; no latency timed from the *publish call*, the moment the producer hands the message over, falls below zero.

- 2) **The mechanism, and its laws** (Section IV). A thread must wait for a core before it can read the clock, and the thread that reads the acknowledgment can wait longer than the message takes to reach the consumer. How often that happens rises with load and depends on where the load is placed, falls as the timed path lengthens, and has the timescale of the scheduler’s slice, which the kernel sets from the CPU count. *Real-time priority*, which runs the threads that read the clock ahead of every thread at the default *normal priority*, cut it 7–80× at unchanged load.
- 3) **The industry-wide audit** (Section V). Seven of the ten tools we read at source dispose of such values without counting them. Ten of the eleven we ran take both timestamps in one process, so they never measure a *one-way latency*, the time from one process to another. On a millisecond clock, a filter that keeps only latencies above zero, a *positivity filter*, also deletes the fastest messages, and no public report we audited says how many samples it kept.
- 4) **What to do** (Section VI). Checks for benchmark authors and readers, each with what it buys where we measured it. Better clock synchronization cures neither problem.

II. HOW A MESSAGE IS TIMED

A producer process publishes messages to a broker at a paced rate, and a consumer process reads them back. Each message crosses two legs (Fig. 1a). Leg 1 carries it to the broker, which writes it to its log and returns an acknowledgment along leg 1, and leg 2 carries it on to the consumer. Three timestamps bracket its path: t_{pub} , read immediately before the publish call; t_{ack} , read when the producer learns that the broker has accepted the message; and t_{recv} , read when the consumer holds it. We call any difference of two timestamps a *span*. Three spans follow:

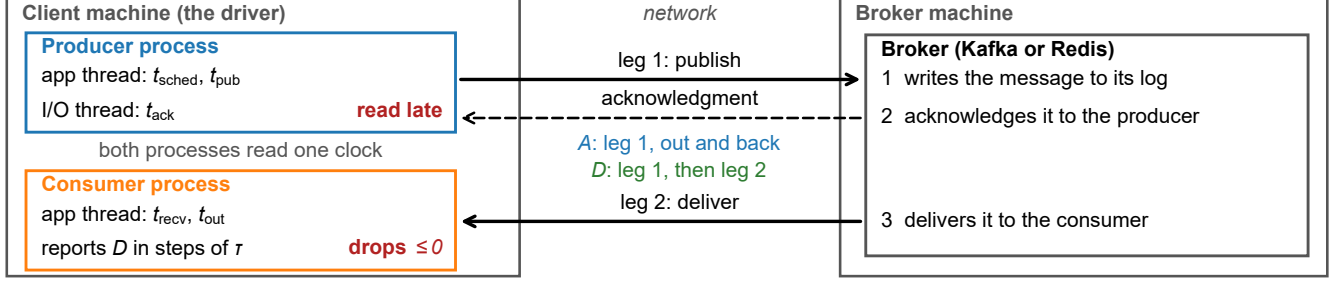
$$\begin{aligned} D &\equiv t_{\text{recv}} - t_{\text{pub}}, & A &\equiv t_{\text{ack}} - t_{\text{pub}}, \\ S &\equiv t_{\text{recv}} - t_{\text{ack}} = D - A. \end{aligned} \quad (1)$$

Broker benchmarks call D the *end-to-end latency*, the message’s time from the publish call to the consumer, and A the *publish latency*, the producer’s wait to hear that the broker has it [3]. S compares the acknowledgment’s return along leg 1 with the message’s trip along leg 2. A benchmark that starts its clock at the acknowledgment, as our own first one did, reports S as the message’s latency.

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.

R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.

(a) Where everything runs



(b) When each timestamp is read

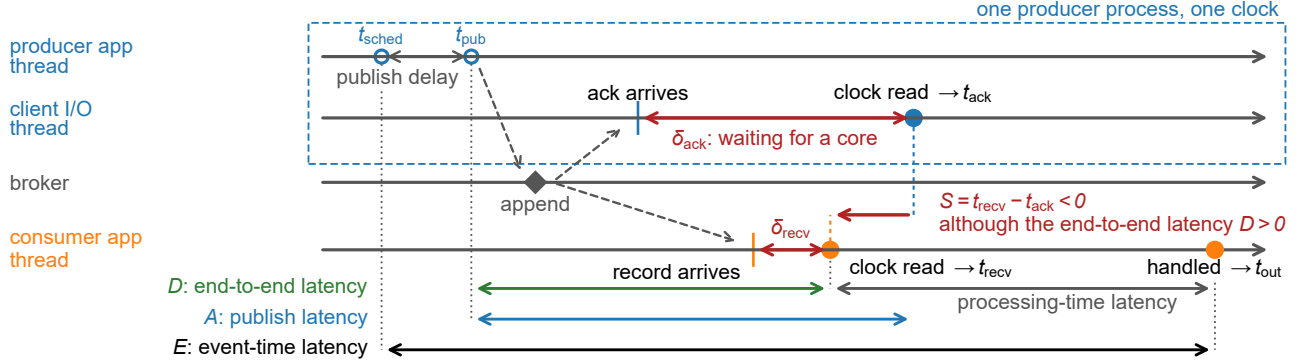


Fig. 1. **Every message crosses two legs, and each timestamp is read after its thread waits for a core.** (a) The producer and consumer processes share a client machine and its one clock, and the broker runs on a second machine. Leg 1 carries a message to the broker, which writes it to its log and returns an acknowledgment; leg 2 carries it on to the consumer. A covers leg 1 out and back, and D leg 1 and then leg 2. (b) The producer reads t_{pub} at the publish call, its client's I/O thread reads t_{ack} only after waiting δ_{ack} for a core, and the consumer reads t_{recv} after its own wait δ_{recv} . Here A runs past D , so $S = D - A$ is below zero although nothing arrived early. The red notes mark this late reading, and a tool that reports D in steps of τ and drops every value at or below zero (Section V). t_{sched} and t_{out} mark when the message was due and when the consumer had handled it, and the *event-time latency* E spans them: D , with the *publish delay* before it and the consumer's *processing-time latency* after it. The path drawn is Kafka's. Redis reads the acknowledgment on the application thread, which also waits for a core.

Unless the text says otherwise, the measurements come from four Oracle Cloud virtual machines on a private network. Three run the brokers, Kafka and Redis Streams. The fourth runs the producer and the consumer as separate CPython processes that read one clock, so nothing here depends on synchronizing clocks. Its kernel is Linux 6.8, whose scheduler is EEVDF (Earliest Eligible Virtual Deadline First). The producer replays StatsBomb's open football event data [4] at a chosen publish rate, and nothing below depends on its content beyond message size and rate. *Utilization* ρ is the fraction of processor time busy on the host over a run, and a *background load*, processor work run beside the clients, sets it. A *condition* is one point of the experimental design: a broker, a publish rate, a message size, a background load and how often the consumer confirms what it has read. A *run* executes a condition once, and a *replicate* is a further run of it. A later campaign ran on three pairs of Microsoft Azure machines, two x86 and one Neoverse N2. It was *registered*: its predictions were committed in public before the runs that test them [5].

III. THE DISTRIBUTION OF S

In the ideal case S is zero, because its two readings follow one write and are taken on one machine and one clock. Measured, it is spread over milliseconds, and part of it falls below zero (Fig. 2). Over the 738,730 messages of the 5,913

cloud runs that produced matched records, 62,264 values of S , 8.43%, fall below zero. We call this share the *negative-span rate*. Both brokers show it, Kafka on 8.67% of its 367,894 messages and Redis on 8.19% of its 370,836. The registered campaign found it again in Kafka's own Java client, at a lower rate, and on Neoverse N2 processors as on x86 (Supplement S6.2).

Nothing arrived early. On the same messages and the same clock, neither D nor A falls below zero once (Supplement S1.3). One reading was taken late. Users of a widely run broker benchmark were told that latencies below zero were clock skew between nodes [6]. On one clock there is no skew to blame.

IV. THE MECHANISM AND ITS LAWS

A thread must wait for a core before it can read the clock. That wait is its *scheduling delay*, the time a thread that is ready to run spends queued before a core runs it, which real-time Linux measures as scheduling latency [7]. We show by *manipulation*, moving one variable with the rest held fixed, that this wait puts S below zero, and state the laws that govern it: when S falls below zero, how often, and on what timescale.

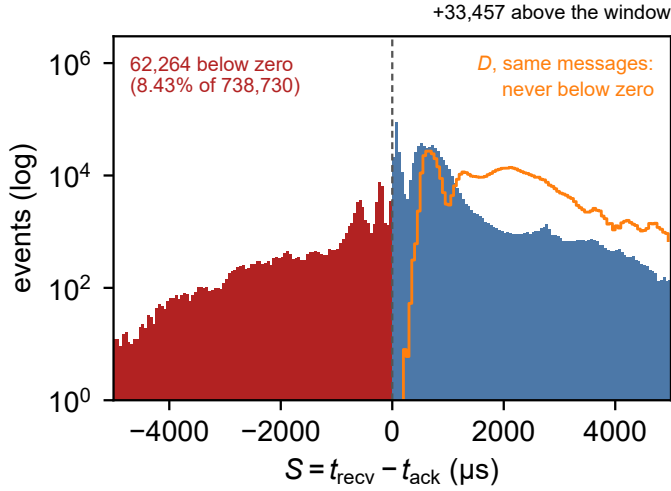


Fig. 2. S spreads over milliseconds, and part of it falls below zero. $S = t_{\text{recv}} - t_{\text{ack}}$ for the 738,730 messages of our cloud runs on both brokers, read on one clock at nanosecond resolution. The 62,264 values below zero are red. The orange line is D , timed from the publish call on the same messages, which is never below zero.

A. When S falls below zero

D and A are causal chains. Each starts at the publish call, which precedes the broker's write, which precedes both the acknowledgment and the delivery. S is not a chain: its two readings are two branches of one cause, the write, and neither has to come first. Since Equation 1 holds for every message, whatever the signs,

$$S < 0 \quad \text{if and only if} \quad A > D. \quad (2)$$

Each reading is late by its thread's wait for a core (Fig. 1b), so S falls below zero when the acknowledging thread's wait, δ_{ack} , outlasts the rest of the message's trip along leg 2 and the consumer's own wait, δ_{recv} .

B. How often, and what moves the rate

Each thread that reads the clock is either running, in which case its reading is late by a narrow jitter, or waiting for a core. From an idle host to the heaviest load before its processors are fully busy, the negative-span rate grows $61\times$ but the jitter's width only $5\times$, so load mostly changes how often the thread is waiting. Write p for that probability, the thread's *waiting probability*, and G for the chance that the rest of its wait, the *residual wait*, outlasts a given time. Write T_{true} for the *true end-to-end latency*, the interval D would report if no reading were late. To leading order,

$$\Pr[S < 0 \mid T_{\text{true}}] \approx p G(T_{\text{true}}), \quad (3)$$

the waiting probability times the chance that a residual wait outlasts the end-to-end latency. Load, its placement and priority move p , and the path's length moves the argument of G .

Priority. Real-time priority for the threads that read the clock should cut their waiting probability while utilization stays fixed. Across eight matched pairs from 60 to 95% utilization, it cut the rate 7- to 80-fold, every pair with disjoint

Wilson intervals and with achieved utilization matched to within 0.002 (Fig. 3b; Supplement S3.1). That refutes any account in which utilization alone sets the rate, including the queueing form we had ourselves adopted. In the registered campaign, on other machines, real-time priority removed the values below zero on Kafka in every run, and load raised the rate in all five Kafka campaigns that tested it (Supplement S6.2).

Placement. With the background load concentrated on six of the eight cores or spread over all of them, two configurations reached the same utilization, $\rho = 0.7531$. Their rates differ 2.07-fold [Katz 95%: 1.80–2.39], and 2.05-fold [Katz 95%: 1.73–2.43] in a full replication (Supplement S3.2). The spread load leaves no core free, so a woken thread waits more often.

Path length. A wait of a given size overlaps a short path almost entirely and a long one hardly at all. Padding the payload lengthens the path without touching the scheduler. It raised the median of S 77-fold while utilization varied by 0.012, and the rate fell 4.1-fold. That is the direction G predicts, and the opposite of what stress on the host would cause (Supplement S3.3).

An independent check. A kernel trace of the scheduler's wakeup and switch events [8] times the wait directly and shares no probe or data with the application's timestamps. With nothing fitted, the share of traced wakeups that waited longer than the end-to-end latency stands to the measured rate at ratios of 0.78, 1.06, 1.32 in three configurations (Supplement S3.5).

C. The timescale the slice sets

A thread woken while another holds the core waits out what is left of that thread's *slice*, the time the scheduler lets a thread run at a turn, up to a whole one. The kernel's documented rule [9] sets the base slice s from the number of CPUs n , with a cap:

$$s = 0.75 \text{ ms} \times (1 + \lfloor \log_2 \min(n, 8) \rfloor), \quad (4)$$

which gives $4 \times 0.75 \text{ ms} = 3 \text{ ms}$ on our hosts' eight CPUs. The traced waits carry that timescale. Over 551,956 wakeups, their counts have three local maxima, which survive a $2\times$ widening of the buckets. The last sits in the 2–4 ms bucket that contains the slice, with 10.5% of all wakeups, $4.5\times$ the bucket below it, and above it the counts collapse (Supplement S3.7). The registered campaign timed the wait acknowledgment by acknowledgment, and a reading made after its verdicts found it to be the rest of a slice (Supplement S6.3).

The CPU count moves the slice. In the registered campaign the slice read back at 2, 4 and 8 CPUs as Equation 4 gives, and on Kafka, at the one CPU count whose slice is at least its shortest end-to-end latency, the end-to-end latency at which the rate falls moved with the slice (Supplement S6.2).

A whole slice is 1.02–4.29 times the per-condition median end-to-end latencies we measured, 0.70–2.95 ms, so the slice lands on the scale of the paths now benchmarked. Read together, Equations 2, 3 and 4 say what the rate depends on. It rises with the waiting probability p , which load raises, a spread load more, and real-time priority lowers. It falls as the

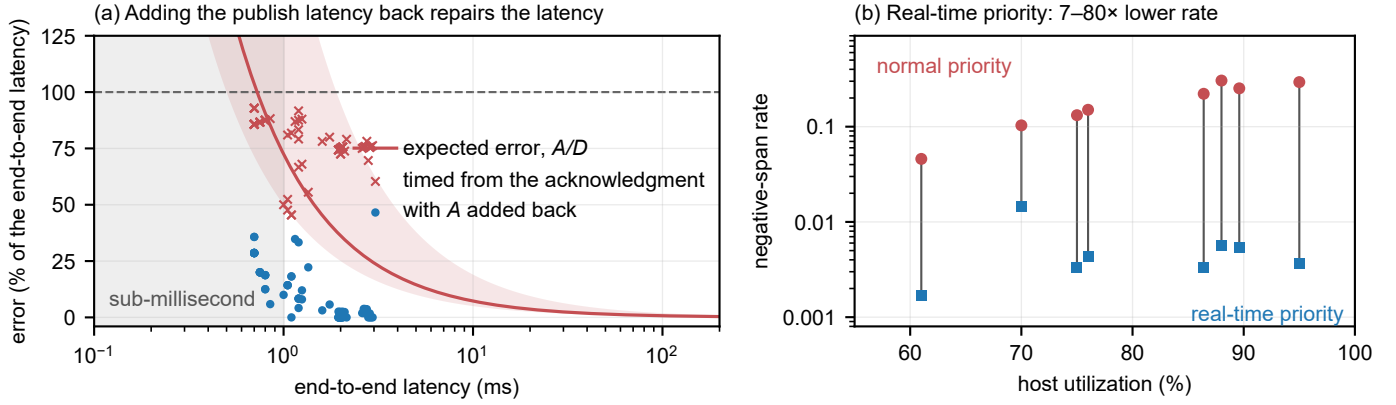


Fig. 3. **Adding the publish latency back repairs the latency, and real-time priority cuts the rate in every pair.** (a) The error of S as a latency on each of our 70 conditions (crosses), and once the publish latency is added back (dots), against the condition’s end-to-end latency. The curve is A/D at the median publish latency, and its band covers the tenth to ninetieth percentiles of A across conditions, 500–1900 μ s. On the dashed line A equals D . The shaded decade holds the sub-millisecond medians now reported for brokers. (b) The negative-span rate of each matched pair at normal priority (circles) and real-time priority (squares), at its own host utilization. Every pair falls, with disjoint intervals.

end-to-end latency outgrows the residual wait, whose scale the slice sets and the CPU count moves. On our kernel, any host with at least as many CPUs as ours has a slice of 3 ms, so a single wait can outlast any end-to-end latency shorter than that.

V. AN INDUSTRY-WIDE AUDIT

What do the industry’s tools do with a latency at or below zero, and with one shorter than their clock’s step? We read ten tools at source, ran eleven under registered predictions, measured one benchmark’s filter in depth and audited the public reports made with it. The four agree. Such a value is deleted, replaced or never measured, and almost never counted.

A. Tools read at source

We read ten measurement tools at source, across five languages and nine vendors, and classified them with one program whose verdicts, each with its source line, are in the supplement’s tool registry (Supplement S5). Seven dispose of a value at or below zero without counting it, in three ways: a filter drops it, the tool’s own library refuses it, or another value replaces it. Replacement hides best, because nothing looks missing: `fib` and `btt` substitute zero and one nanosecond, and RabbitMQ PerfTest takes the absolute value [10]. Apache Kafka’s own end-to-end tool rounds each latency down to whole milliseconds, so on a path faster than a millisecond every percentile it reports is zero [11]. Only `emqtt-bench`, which keeps every sample [12], and Rezolus, which counts its discards [13], keep the evidence.

B. Tools run

Reading a tool shows what it does with a value below zero, not whether it ever meets one. The registered campaign therefore ran eleven benchmarking tools against a broker on a second machine, four rounds each, with predictions fixed from their source before any run (Supplement S5). A delay added to the path in steps moved every tool’s figure 0.92–1.14 ms per

millisecond added, for each time the interval it times crosses the delay, so every tool measures the path. Moving back the clock a tool reads, by more than the interval it times, separated the tools that read one clock from those that read two. The figures of ten tools did not move in 70 of their 80 runs, the rest undecided, because each takes both timestamps in one process, timing a round trip or a message it publishes and receives itself. Such a tool never meets a value below zero, and for the same reason it cannot measure the one-way latency of the delivery a broker exists to provide. The eleventh, `librdkafka’s rdkafka_performance`, timestamps in the producer and subtracts in the consumer. Its figures moved with the clock, and in three of its four rounds it kept every value, those below zero included, in its average, so its figure adds the clocks’ offset to the path’s latency.

C. A positivity filter on a millisecond clock

The positivity filter is one of the dispositions above, and the one that deletes more than the values below zero. In statistical terms it truncates the distribution at zero: what it removes is not counted, as it would be if censored. We measured it in depth in a broker benchmark that many public comparisons run, the OpenMessaging Benchmark [3]. Its filter was added in 2018 because its end-to-end latency “is calculated based on time difference of `System.currentTimeMillis()` on two different machines” and “can be negative” [14], and its histogram rejects values below zero outright. For clocks that disagree, the filter is a sound local fix.

It has two side effects. Its clock counts whole milliseconds, so its *timestamp resolution* τ , the smallest step a timestamp can take, is a millisecond, and a message that arrives before the millisecond digit changes computes to zero and is deleted. And nothing counts what the filter deletes. A sample survives only if a millisecond boundary falls between its two timestamps. With publish instants placed uniformly within the millisecond, that happens with probability T_{true}/τ on a path shorter than

τ , so the share of samples kept, the *retention*, is

$$\text{retention} = \min\left(1, \frac{T_{\text{true}}}{\tau}\right). \quad (5)$$

Every sample kept on such a path reads exactly one step, so two brokers whose messages all arrive within a millisecond print the same median and differ only in their retention, which is never printed.

We added counters that record, by sign, every sample the filter discards, and changed nothing else. Every run was in the benchmark's embedded mode, which runs it whole in one Java virtual machine. Of the 75 settings whose own summary we captured, 71 printed a median of exactly 1.0 or 2.0 ms, computed from 0.36–100% of the samples taken. Two replicates of one setting kept 431 and 120,434 samples, 279-fold apart, and printed the same median, with nothing in the output to tell them apart. The filter can also inflate a median. At 64 KB, the two settings kept only 34.4–35.9% of their samples, and every sample they deleted computed to zero, so most of their messages arrived within a millisecond. Yet they printed medians of 235 and 519 ms, those of the slow remainder.

Equation 5 holds where it can be tested. All seven configurations whose publish interval bears no ratio of small whole numbers to τ keep 47.2–51.5%, which puts T_{true} at 0.49 ms. Where the ratio is one of small whole numbers, the periodic schedule phase-locks with the clock: the publish instants visit only a few positions within the millisecond, and retention takes only a few values. A registered manipulation of message size moved one configuration onto one of those values and off it (Supplement S4.3). The arithmetic is in Hewlett-Packard's 1970 counter note [15], but the deletion is not: the counter keeps its zero readings and estimates the interval from the share that are not zero.

Signed counters show the filter hiding two different failures. On Kafka, the 10,913,263 samples it discarded over 214 runs hold 0 values below zero, so every discard is a loss of resolution. On Redis, the benchmark subtracts a millisecond timestamp the broker assigns from the consumer's millisecond clock on another host. All 41,403 values below zero over its 9 runs are exactly $-\tau$, the only value below zero that two floored clocks less than a millisecond apart can produce (Supplement S4.4). An unsigned counter cannot tell the two failures apart.

D. Public reports

We registered an audit of the benchmark's public reports before reading any (Supplement S5). Its signature is a printed end-to-end median of exactly 1 ms, the smallest value the benchmark can print and the value every kept sample takes on a path faster than a millisecond. Of 43 public reports that print the benchmark's end-to-end percentiles, across 1,100 configurations, eight of the 868 medians they print carry it, in two reports. None of the reports states how many samples it kept. The signature is rare partly because 14 of the reports come from copies of the benchmark whose maintainers changed how the span is timed. They print medians below

TABLE I
MOST STEPS COST LITTLE, AND MOST WERE MEASURED. WHAT A BENCHMARK AUTHOR OR READER SHOULD DO, AND WHAT EACH STEP BUYS ON OUR RUNS.

Do this	What it buys
Sign-check every run under the load it is quoted at, and report the rejection rate.	Rejected 51.9% of our cloud runs. Sees only the messages whose A exceeds D .
Time from the publish call, or record the publish latency and add it back.	Recovers D to a median error of 3.4% where the sign check passes and 12.2% where it fails. S alone understates D $4.2\times$ at the median (Fig. 3a).
Give the threads that read the clock real-time priority, or have them spin on a dedicated core (<i>busy-polling</i>).	$7\text{--}80\times$ fewer values of S below zero, in each of eight matched pairs (Fig. 3b). Busy-polling: not measured.
Record at native resolution, and print the retention and the discards counted by sign beside every latency.	Shows what a filter deleted, and whether the resolution or a clock failed (Section V-C).
Read your own percentiles: on a fast path, a median of exactly one or two clock steps was set by the clock.	Flags 71 of the 75 settings we captured (Section V-C).
Randomize the publish instants (Poisson sampling), and record the achieved rate.	Keeps the schedule from phase-locking with the clock. Not measured.

a millisecond, which the original benchmark's clock cannot produce.

VI. WHAT TO DO

The late reading and the deletion are cheap to detect, and most of what they cost is cheap to repair (Table I).

Sign-check every run. The *sign check* rejects a run when more than one percent of its messages carry a value below zero in any latency component it records, S among them, or when the median of any component is below zero. It rejected 459 of the 884 cloud runs behind our results (51.9%; Supplement S2). It sees only the worst cases. The publish latency exceeds half the end-to-end latency on 74.5% of messages, but the check sees only the 8.4% on which it exceeds the whole, and a latency that load inflates but leaves positive passes by construction. The check therefore needs the repair that follows.

Time from the publish call, or add the publish latency back. The publish latency does not grow because a message travels further, so the relative error of S as a latency, A/D , falls as $1/D$. At our median publish latency, $A = 725\mu\text{s}$, the error is 72% at an end-to-end latency of 1 ms and 7% at 10 ms, and below 0.72 ms the median publish latency exceeds the end-to-end latency itself (Fig. 3a). Across the band of A in the figure, the error at 10 ms runs from 5 to 19%. Over our 70 conditions, S understates the end-to-end latency $4.2\times$ at the median, with an interquartile range of $4.0\text{--}7.5\times$. Because the two brokers' clients read the acknowledgment differently, S also inflates the gap between them $1.7\times$ over 24 matched workloads, a property of the clients that a reader would credit to the brokers.

The repair is Equation 1. $D = S + A$ holds for every message, and A is read on the producer's side of the same clock, so a benchmark that records its publish latency recovers the end-to-end latency it failed to measure. The median error

of that recovery is 3.4% on the 42 conditions the sign check passes and 12.2% on the 28 it rejects, with upper quartiles of 20% and 19%. Timing from the publish call removes the values below zero, but not the consumer's own wait for a core, which lengthens every latency and which no check sees. Measured three ways in 51 later runs on the registered campaign's machines, that wait added 0.41–0.94 ms to the mean end-to-end latency at 75 and 88% background load, while the median message waited 18–46 μ s (Supplement S3.8).

Give the threads that read the clock a core. Real-time priority cut the negative-span rate 7–80 $\times$ at unchanged background load (Fig. 3b). A busy-polling thread spins on a core of its own instead of sleeping, and never waits to be woken. Busy-polling a dedicated core should buy the same for the price of that core, though we did not measure it. At real-time priority the rate settles at 0.0017–0.0144, not at zero, so the check stays, and a real-time or polling thread changes the system it measures.

A better-synchronized clock repairs neither the late reading nor the deletion: the values of S below zero arise on one clock, and the deletion from the clock's step, not its offset. A finer clock confines the deletion to faster paths but leaves the filter and every value of S below zero in place. The remedies are therefore ordered: record at native resolution, count what is discarded, randomize the publish instants, and only then does synchronization quality matter.

VII. RELATED WORK

Scheduling delay. That a ready thread can wait for a core is well known [8], scheduler studies find that wait roughly proportional to the time slice [16], and the limits of the timers that benchmarks read are known too [17]. Load testers of microsecond-scale services correct for the *client bias* their own queueing and scheduling add to the latencies they report [18], [19], Lancet by busy-polling. Chandrasekar and Kramberger model that bias with a single-parameter queueing law [20] that our placement contrast refutes. Villain et al. found that an outgoing socket timestamp “does not respect causality” under every stress pattern, including none, and traced host latency to interrupts, scheduling and context switching [21]. None gives the consequence for a difference read by two processes: that it falls below zero at a rate set by the waiting probability and the path's length, on the timescale of the slice.

Clocks. Elsewhere, values below zero are read as clocks that disagree. Sharma et al. count negative timing spans in distributed inference systems and read them as clock skew, holding that “queueing alone cannot produce negative timing spans” [22]. For the backlog they model we agree, but a timestamping thread waits in a second queue, the kernel's run queue. Van Dongen and Van den Poel, who saw latencies below zero on up to a tenth of their events when timestamps came from different brokers, took both from one broker [23], the one-clock rule reached by another route. Better measurement systems synchronize clocks [24], calibrate counters across nodes [25] or timestamp in the network interface [19] or the switch [26], and the checks of Section VI apply to whatever they measure.

Resolution and discarded samples. The retention arithmetic and its cure, phase jitter, are in a 1970 counter note [15], and IP measurement advises randomizing probe times [27], [28]. Coordinated omission [29] is the deletion's mirror image: it fails to record samples that would have been slow, while a positivity filter deletes samples that were fast. Network measurement reports the share of traces it rejects [30] and keeps a packet timestamped in the future [31], while the literature on rigorous benchmark reporting [32], [33], [34] does not check what a tool discards, nor do the principles for reproducible performance evaluation in the cloud [35]. Research benchmarks of stream processing compute latency from two broker-assigned millisecond timestamps [36], so the retention arithmetic applies wherever their latencies fall below a millisecond. What we add is these checks, applied where a benchmark reports a latency.

VIII. LIMITATIONS

Attributing the values of S below zero to scheduling is an inference. One clock excludes clock skew, the manipulations exclude utilization, timer noise and stress, and the kernel trace observes the wait. Clocks that disagree across virtual CPUs [37] are excluded too. A clock read in 90 ns can only be `kvm-clock` or `tsc`, both of which the kernel uses only where the CPUs' clocks agree, and an offset large enough for the deepest value of S would have driven D below zero, while the smallest D observed caps any offset 455-fold lower. Both testbeds are virtual, so *steal time*, the time a virtual processor is ready but waits while the hypervisor runs another machine, is bounded, not excluded: real-time priority, a setting inside our virtual machine, cannot shorten it, yet it cut the rate. Our CPython timestamps also wait on the interpreter lock, but Kafka's Java client, which has none, shows the same values below zero.

The laws hold to leading order. A registered prediction that priority would cut the waiting probability at least tenfold failed, scheduler counters seeing it fall 1.7- to 2.2-fold, so the factorization of Equation 3 is untested. The tracer is not free: the rate was 0.272 untraced against 0.231 traced ($z = 3.6$), so each prediction is held against the traced run's own rate, and a check fixed in advance set aside one of the three configurations; the other two bracket the observed rate. The base slice on the cloud hosts is derived, not read off them. In the registered campaign, the predictions of where the rate falls with the end-to-end latency, and of how wide the fall is, were not confirmed, and Python's rate exceeded the registered limit of 1.5 times Java's (Supplement S6). The constants are those of one EEVDF-era kernel. Designs that pin a busy-polling thread to a dedicated core, or timestamp in hardware, remove the waiting state, and we make no claim about them.

The deletion is demonstrated in one benchmark, in its embedded mode, and reproduced in one independent program of our own; the benchmark's distributed mode failed on every attempt (Supplement S4.5). One registered detail failed: the 32 KB configuration pins at 69.2% retention, not the predicted 66.67%. The audit found the deletion's signature in few public results, so we claim it for those and our own runs only. We

classified the ten tools by reading them, and ran the eleven on one pair of machines, one kernel and one publish rate. How much a deployment loses depends on its end-to-end latency and publish rate, so we recommend reporting retention rather than calling any reported number wrong.

IX. CONCLUSION

A latency benchmark measures the system only while its readings are taken faster than the latency it measures. On one clock, the producer's reading of an acknowledgment and the consumer's reading of the message, which should coincide, differ by milliseconds and in either order, because a thread waits for a core, for up to a scheduler's slice, before it reads the clock. Across the industry such values are deleted, replaced or never measured, a positivity filter on a millisecond clock deletes the fastest messages too, and no report we audited says how many samples it kept. Before acting on a broker latency below a few milliseconds, ask which timestamp it was timed from, how many samples the tool kept, and how many it discarded, by sign.

ACKNOWLEDGMENT

The authors thank N. Herbst, for the load-generation point that became one of the checks above, for pointing us to the timer study cited in Section VII, and for a reading that reset the abstract.

In accordance with IEEE policy, Anthropic's Claude assisted with code development and verification, manuscript editing, and literature search. Experimental design, scientific claims, and interpretation are the authors'.

ARTIFACT AVAILABILITY

Code, analysis and manuscript are archived at 10.5281/zenodo.21650031, the measurement dataset at 10.5281/zenodo.21650064; both resolve to the current version, v4.0.0, whose files are restricted pending publication. Every number here is recomputed from the committed data by the archived code at build time, and the test suite fails if the text and the data disagree.

REFERENCES

- [1] P. Dobbelaere and K. S. Esmaili, "Kafka versus RabbitMQ: A comparative study of two industry reference publish/subscribe implementations," in *Proc. 11th ACM Int. Conf. Distributed and Event-Based Systems (DEBS)*. ACM, 2017, pp. 227–238.
- [2] Index.dev, "NATS vs Redis vs Kafka: message broker comparison," Practitioner comparison, 2026, <https://www.index.dev/skill-vs-skill/nats-vs-redis-vs-kafka>. Accessed: Sep. 3, 2026.
- [3] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.
- [4] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023. [Online]. Available: <https://github.com/statsbomb/open-data>
- [5] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Natl. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, 2018.
- [6] OpenMessaging Benchmark contributors, "Can the average publish latency be larger than end-to-end latency? (issue #216)," <https://github.com/openmessaging/benchmark/issues/216>, 2021.
- [7] D. Bristol de Oliveira, D. Casini, J. Lelli, and T. Cucinotta, "Timerlat: Real-time Linux scheduling latency measurements, tracing, and analysis," *IEEE Trans. Comput.*, vol. 74, no. 8, pp. 2608–2620, 2025.
- [8] B. Gregg, "Linux bcc/BPF run queue (scheduler) latency," <https://www.brendangregg.com/blog/2016-10-08/linux-bcc-runqlat.html>, Oct. 2016, accessed: Sep. 13, 2026.
- [9] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit 2ae891b82695, 2025.
- [10] Broadcom, "RabbitMQ PerfTest — TimestampProvider.java," Source repository, 2026, <https://github.com/rabbitmq/rabbitmq-perf-test>. Accessed: Oct. 2, 2026.
- [11] Apache Software Foundation, "Apache Kafka tools — EndToEndLatency.java and ProducerPerformance.java," Source repository, 2026, <https://github.com/apache/kafka>. Accessed: Aug. 31, 2026.
- [12] EMQ Technologies, "emqtt-bench: MQTT benchmark tool — src/emqtt_bench.erl," Source repository, 2026, <https://github.com/emqx/emqtt-bench>. Accessed: Aug. 21, 2026.
- [13] IOP Systems, "Rezulus: systems performance telemetry — scheduler_discarded_samples," Source repository, 2026, <https://github.com/iopsystems/rezulus>. Accessed: Aug. 20, 2026.
- [14] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, <https://github.com/openmessaging/benchmark/pull/56>. Accessed: Aug. 18, 2026.
- [15] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766.
- [16] Y. Jia, K. Tian, Y. You, Y. Chen, and K. Chen, "Skyloft: A general high-efficient scheduling framework in user space," in *Proc. 30th ACM Symp. Operating Systems Principles (SOSP)*, 2024, pp. 265–279.
- [17] M. Kuperberg, M. Krogmann, and R. H. Reussner, "Metric-based selection of timer methods for accurate measurements," in *Proc. 2nd ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, Mar. 2011, pp. 151–156.
- [18] Y. Zhang, D. Meisner, J. Mars, and L. Tang, "Treadmill: Attributing the source of tail latency through precise load testing and statistical inference," in *Proc. 43rd ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2016.
- [19] M. Kogias, S. Mallon, and E. Bugnion, "Lancet: A self-correcting latency measuring tool," in *Proc. USENIX Annual Technical Conf. (ATC)*, 2019, pp. 881–896.
- [20] A. Chandrasekar and J. Kramberger, "Identifying and mitigating systemic measurement bias in production LLM inference benchmarks," *arXiv preprint arXiv:2605.24217*, 2026.
- [21] B. Villain, M. Davis, J. Ridoux, D. Veitch, and N. Normand, "Probing the latencies of software timestamping," in *Proc. IEEE Int. Symp. Precision Clock Synchronization for Measurement, Control and Communication (ISPCS)*, 2012, pp. 1–6.
- [22] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, Open Compute Project, Unified Intelligent Infrastructure workstream.
- [23] G. Van Dongen and D. Van den Poel, "Evaluation of stream processing frameworks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 8, pp. 1845–1858, 2020.
- [24] IEEE, "IEEE standard for a precision clock synchronization protocol for networked measurement and control systems," *IEEE Std 1588-2019*, 2020, revision of IEEE Std 1588-2008.
- [25] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.
- [26] R. Kundel, "P4STA: High precision network function benchmarking," in *Accelerating Network Functions Using Reconfigurable Hardware*, ser. Springer Theses. Springer Nature Switzerland, 2024, pp. 93–115.
- [27] V. Paxson, G. Almes, J. Mahdavi, and M. Mathis, "Framework for IP performance metrics," Internet Engineering Task Force, RFC 2330, May 1998.
- [28] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.
- [29] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop / QCon.
- [30] V. Paxson, "On calibrating measurements of packet transit times," in *Proc. ACM SIGMETRICS*. ACM, 1998, pp. 11–21.
- [31] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.
- [32] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.

- [33] A. Georges, D. Buytaert, and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOP-SLA)*. ACM, 2007, pp. 57–76.
- [34] T. Hoefler and R. Belli, “Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results,” in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.
- [35] A. V. Papadopoulos, L. Versluis, A. Bauer, N. Herbst, J. von Kistowski, A. Ali-Eldin, C. L. Abad, J. N. Amaral, P. Tüma, and A. Iosup, “Methodological principles for reproducible performance evaluation in cloud computing,” *IEEE Trans. Softw. Eng.*, vol. 47, no. 8, pp. 1528–1543, 2021.
- [36] S. Henning, A. Vogel, M. Leichtfried, O. Ertl, and R. Rabiser, “ShuffleBench: A benchmark for large-scale data shuffling operations with distributed stream processing frameworks,” in *Proc. 15th ACM/SPEC Int. Conf. Performance Engineering (ICPE)*, 2024, pp. 2–13.
- [37] Linux Kernel Documentation, “KVM-specific MSRs: MSR_KVM_SYSTEM_TIME_NEW,” <https://docs.kernel.org/virt/kvm/x86/msr.html>, 2026, accessed: Aug. 21, 2026.

Gustavo Pedro Ricou received the B.Sc. degree in physics from the Universidade do Porto, Portugal, and specialized in nuclear fusion science at the Universität Stuttgart, Germany, and the Université de Lorraine, France. He holds an M.Sc. in software design with cloud-native computing from the Technological University of the Shannon, with first-class honors, and is currently pursuing an M.Sc. in statistics and data science at Trinity College Dublin, Ireland. He is a Software Engineer with BoscaSports. His research concerns measurement validity — what a reported number is really evidence of — in computer systems, environmental measurement and sports analytics, using pre-registration and physical-consistency auditing.

Romarie Duvignau received the Ph.D. degree in computer science from the University of Bordeaux, France, in 2015, for work on the maintenance and simulation of dynamic random graphs. He is an Associate Professor and Head of the Network and Systems unit at Chalmers University of Technology, Gothenburg, Sweden. His research concerns distributed and data-driven algorithms, the monitoring of large-scale distributed systems, and stream analytics for cyber-physical systems.